INSTITUTO FEDERAL DE GOIÁS PÓS-GRADUAÇÃO EM INTELIGÊNCIA ARTIFICIAL APLICADA

ANA BEATRIZ PAES BORGES
RODOLFO MELO NASCIMENTO PIRES
WEDERSON DOMINGUES

RELATÓRIO REDES RECORRENTES - TAXA DE APROVAÇÃO EM ESCOLAS

GOIÂNIA
2026

1 INTRODUÇÃO

A taxa de aprovação é um dos indicadores que ajudam a entender como está o desempenho das escolas ao longo dos anos. Observar a evolução desse indicador permite identificar mudanças no comportamento dos dados e levantar questões sobre os fatores que podem influenciar os resultados educacionais. A partir disso, surgiu a ideia de investigar se seria possível utilizar técnicas de inteligência artificial para aprender com o histórico das escolas e prever suas taxas de aprovação em anos seguintes.

Neste trabalho, buscamos utilizar redes neurais recorrentes para prever a taxa de aprovação do primeiro ano do ensino fundamental em escolas brasileiras. Para isso, utilizamos uma base de dados disponibilizada no Portal Brasileiro de Dados Abertos, que reúne informações de diferentes escolas ao longo de vários anos. Cada escola foi tratada como uma sequência de dados, permitindo analisar como suas taxas de aprovação se comportaram historicamente e se esse comportamento poderia ajudar a estimar um resultado futuro.

Durante o desenvolvimento, foram realizados diferentes experimentos para entender quais configurações apresentavam os melhores resultados. Inicialmente, utilizamos os dados de 2014 a 2018 para prever a taxa de aprovação de 2019. Em seguida, testamos diferentes formas de preparação dos dados, incluímos informações sobre a região e a rede de ensino das escolas e ampliamos o período analisado. Também comparamos três arquiteturas de redes recorrentes: SimpleRNN, LSTM e GRU, buscando identificar se alguma delas apresentava vantagens para esse tipo de previsão.

Além de avaliar os resultados das redes, procuramos compará-los com estimativas mais simples, baseadas na média, na mediana e no último valor observado. Essa comparação foi importante porque um erro de previsão baixo, isoladamente, não significa necessariamente que o modelo aprendeu padrões relevantes nos dados. Como as taxas de aprovação apresentam uma concentração expressiva de valores próximos de 100%, foi necessário considerar essa característica na interpretação dos resultados.

Assim, o objetivo deste trabalho é verificar se as redes neurais recorrentes conseguem utilizar o histórico das escolas para prever suas taxas de aprovação e se apresentam resultados melhores do que métodos de referência mais simples. Com os experimentos realizados, buscamos compreender quais escolhas influenciam o desempenho das previsões e se modelos mais complexos realmente oferecem vantagens para o problema estudado.

2 BASE DE DADOS

A base de dados foi extraída do portal de dados abertos do governo federal e consiste em indicadores de escolas de todo Brasil dos níveis de ensino do início do fundamental ao final do ensino médio.

A base consiste em valores dados desde 2007 até 2025, assim tendo sequências de valores suficientes para atender o requisito para esse projeto, onde cada escola foi considerada uma sequência diferente.

A disponibilização dos arquivos não foi feita de forma homogênea, contendo arquivos no formato XLS e XLSX, onde o padrão de pastas também seguiam de forma diferentes entre os anos.

Para solucionar esse problema foi utilizado um script python, gerado por IA, para unificar as bases em um arquivo CSV único contendo todos os registros.

Dos dados disponibilizados foram escolhidos os seguintes como features:

-REGIAO: Macrorregião da escola, exemplo de dados: Nordeste, Norte, Sudeste etc.

-Rede: Federal, Estadual etc

-ANO: Ano referente as taxas

-Aprovação no 1º Ano do Ensino Fundamental

Vale destacar que apesar do ensino fundamental ser apenas da rede municipal, ainda existem escolas das redes federais e estaduais que possuem essa modalidade de ensino, estando elas em fase de transição para o modelo municipal.

Após a unificação das bases, foi separado 15% dos dados das escolas em um arquivo e o restante em outro arquivo, sendo o primeiro dado reservados para a etapa de teste, e os demais para as etapas de treino e validação. O segundo dataset foi dividido em 70% para treino e 30% para validação.

3 RELATÓRIO

Dentro de um ambiente Google Colab foi utilizado a biblioteca Pandas para fazer a primeira carga de dados de treino e validação e a seleção dos indicadores que poderiam ser utilizados.

Figura 1 – Trecho de código utilizado para carregar e selecionar os dados da base no ambiente Python

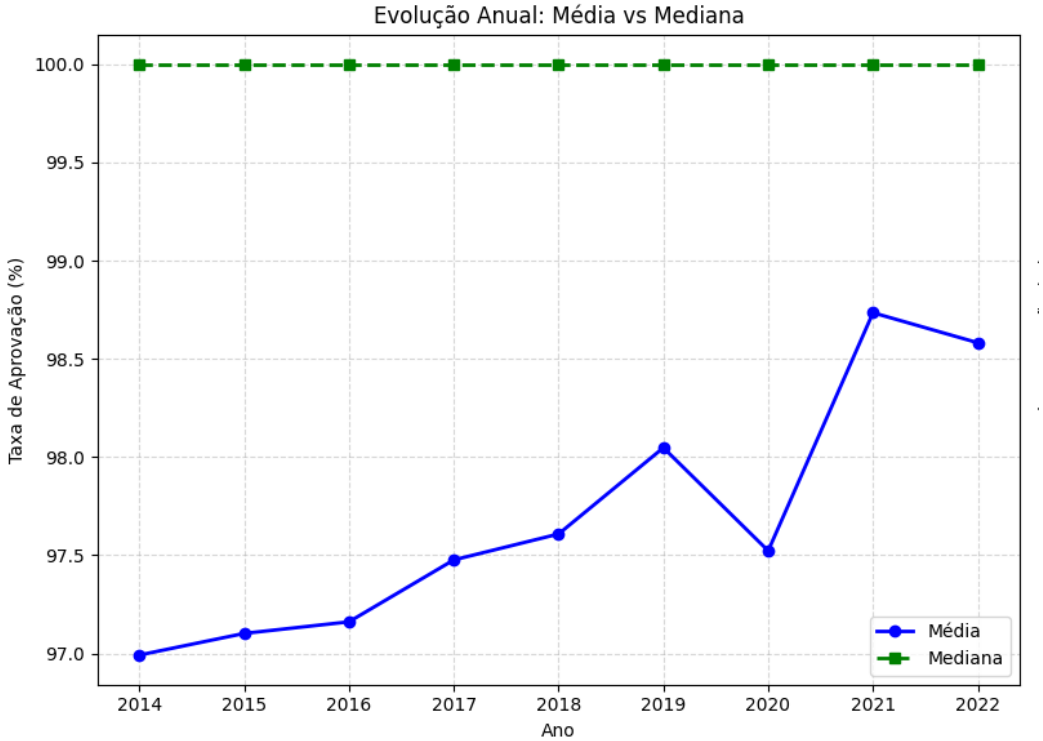
```
dataframe_raw = pandas.read_csv('ensino_unificado.csv', delimiter=';', usecols=["ANO","REGIAO","UF","CÓDIGO DO MUNICIPIO","REDE","CÓDIGO DA ESCOLA","Aprovação no 1º Ano do Ensino Fundamental"])
```

Fonte: Elaborado pelos autores (2026).

Os primeiros testes foram realizados filtrando os dados entre 2014 e 2019, evitando pegar anos de relatórios muito antigos e também não incluindo anos de pandemia que poderiam gerar uma distorção na base.

Foi decidido que os anos entre 2014 e 2018 seriam features para a rede e o de 2019 o target. Na imagem abaixo é possível ver a quebra no padrão da sequência em 2020.

Figura 2 – Gráfico da evolução anual da média e da mediana das taxas de aprovação



Fonte: Elaborado pelos autores (2026).

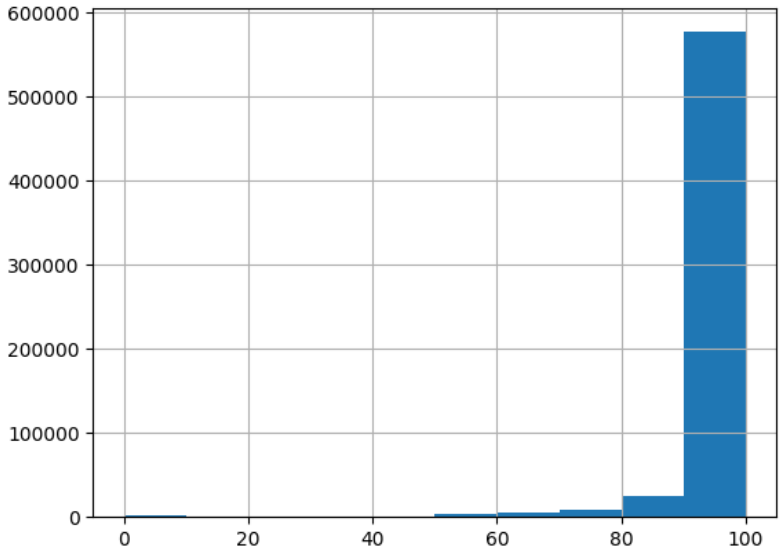
Com a base carregada e timeline determinada, a etapa seguinte foi a exploração dos dados para entender o comportamento da base. Para isso foi utilizado as ferramentas de histograma, análise de métricas estatísticas, um gráfico de linha e um gráfico boxplot.

Figura 3 – Tabela de estatísticas descritivas anuais, com média, mediana e desvio-padrão

	ANO	mean	median	std
0	2014	96.991525	100.0	9.671758
1	2015	97.101648	100.0	9.562904
2	2016	97.160722	100.0	9.591214
3	2017	97.475840	100.0	8.819606
4	2018	97.607891	100.0	8.450502
5	2019	98.047225	100.0	7.656682

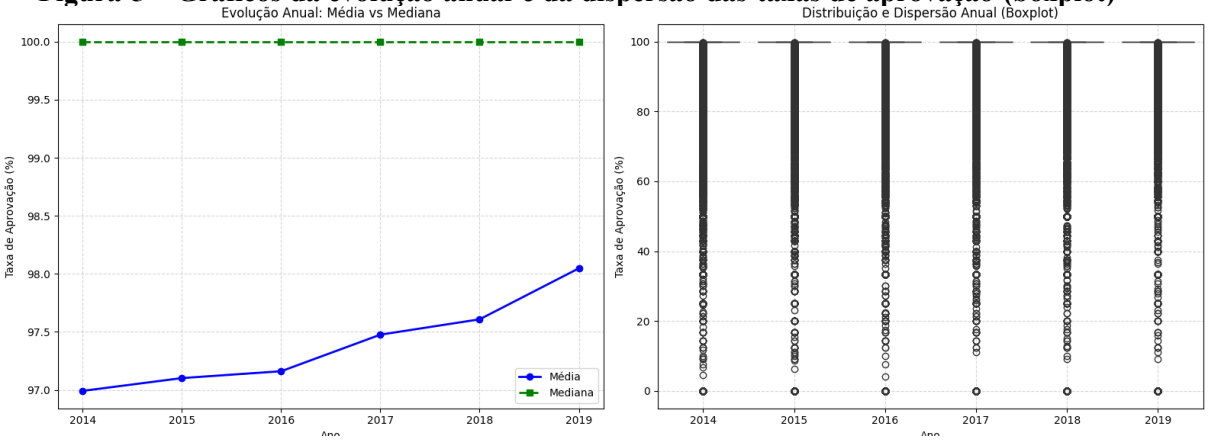
Fonte: Elaborado pelos autores (2026).

Figura 4 – Histograma da distribuição das taxas de aprovação



Fonte: Elaborado pelos autores (2026).

Figura 5 – Gráficos da evolução anual e da dispersão das taxas de aprovação (boxplot)



Fonte: Elaborado pelos autores (2026).

A análise dos indicadores mostrou uma distribuição assimétrica à esquerda, dada por uma alta concentração de valores próximos a taxa de 100%. Como o indicador representa uma taxa de aprovação, essa concentração pode ocorrer na base analisada. Por isso, valores distantes da concentração principal devem ser interpretados com cuidado: eles podem representar situações reais das escolas e não necessariamente erros de registro.

Com a exploração feita o próximo passo foi a realização do primeiro teste que consiste em uma **normalização escalar entre 0 e 1**, dividindo o indicador por 100, visto já serem percentuais.

O processo de limpeza da base consistiu na remoção de séries com features ausentes, como escolas sem registro em algum dos anos da série histórica.

Após a limpeza a base consistia em **35.169 séries distintas**.

Foi utilizada a função keras.utils.split_dataset para dividir os dados em treino e validação, destinando 70% à primeira partição e 30% à segunda. Os dados foram divididos em batchs de tamanho 24.

Figura 6 – Trecho de código que organiza as sequências anuais e separa os dados em treino e validação

```
dados = dataframe_pivotado[[2014, 2015, 2016, 2017, 2018, 2019]].to_numpy(dtype="float32")

# Features: 2014-2018
X = dados[:, :-1]

# Target: 2019
y = dados[:, -1:]

X = np.expand_dims(X, axis=-1)

dataset_train, dataset_test = keras.utils.split_dataset(
    (X, y),
    left_size=0.7,
    shuffle=True,
    seed=42
)

dataset_train = dataset_train.batch(24)
dataset_test = dataset_test.batch(24)
```

Fonte: Elaborado pelos autores (2026).

A rede escolhida foi uma **simpleRNN** com **5 neurônios** na camada inicial e **16 neurônios** na camada densa com apenas 1 neurônio na camada de saída, a função de ativação **linear** na camada de saída também foi a primeira a ser testada. O treinamento ocorreu em apenas **4 épocas**.

Como métricas de assertividade para a rede, foi utilizado o **mean_absolure_error** e a própria **loss** de rede.

Figura 7 – Registro das métricas de treinamento e validação da rede SimpleRNN ao longo das épocas

```
history = model.fit(
    dataset_train,
    validation_data=dataset_test,
    epochs=4
)
```

```
Epoch 1/4
1763/1763 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - loss: 0.0144 - mean_absolute_error: 0.0466 - val_loss: 0.0051 - val_mean_absolute_error: 0.0292
Epoch 2/4
1763/1763 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 0.0046 - mean_absolute_error: 0.0296 - val_loss: 0.0050 - val_mean_absolute_error: 0.0295
Epoch 3/4
1763/1763 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 0.0046 - mean_absolute_error: 0.0300 - val_loss: 0.0049 - val_mean_absolute_error: 0.0274
Epoch 4/4
1763/1763 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - loss: 0.0046 - mean_absolute_error: 0.0300 - val_loss: 0.0049 - val_mean_absolute_error: 0.0282
```

Fonte: Elaborado pelos autores (2026).

Foi percebido que já na primeira iteração foi atingida uma loss muito baixa tanto na fase de treino quanto na fase de validação, mostrando que a rede **não apresentou Overfitting**.

Após algumas análises foi concluído que isso se deu justamente pela distribuição da rede, com uma grande **concentração de valores em 100**. Qualquer número que a rede forneça

entre 95-100 estaria muito próximo da resposta, seja ela qual fosse. Não necessariamente indicando que a rede estaria apresentando bons resultados.

Dado esse desafio, o grupo decidiu utilizar os indicadores de MAE, RMSE e R^2 porém não apenas da rede nas épocas, mas comparando com os indicadores de **mediana do treino**, **média do treino**, **média dos últimos 5 anos e** valor absoluto do último ano antes do target.

A ideia passou a ser não apenas otimizar a rede para reduzir a loss, mas a rede conseguir encontrar padrões na sequência que descrevam os valores melhor que os demais indicadores, a pergunta então vira '**Conseguimos utilizar redes recorrentes para prever a taxa de aprovação do primeiro ano do ensino fundamental melhor do que a média e mediana da sequência?'**

Figura 8 – Tabela comparativa da SimpleRNN e dos métodos de referência, com MAE, RMSE e R^2

	Modelo	MAE (p.p.)	RMSE (p.p.)	R^2
0	Mediana do treino	1.9429	7.5222	-0.0715
1	RNN	2.3325	6.8774	0.1043
2	Último ano (2018)	2.7946	9.1560	-0.5875
3	Média dos 5 anos	2.9471	7.4624	-0.0545
4	Média do treino	3.1292	7.2674	-0.0001

Melhoria da RNN sobre o baseline '2019 = 2018': 16.54%

Fonte: Elaborado pelos autores (2026).

Com a definição dos testes, o próximo passo foi alterar a função de ativação da camada de saída para **tanh**, buscando melhorar o resultado. Entretanto, o desempenho foi inferior ao obtido anteriormente: o MAE passou de 2,3325 para 2,9182 pontos percentuais. Nas condições avaliadas, essa alteração não trouxe melhoria para a previsão.

Figura 9 – Tabela de resultados após a alteração da função de ativação para tanh

	Modelo	MAE (p.p.)	RMSE (p.p.)	R²
0	Mediana do treino	1.9429	7.5222	-0.0715
1	Último ano (2018)	2.7946	9.1560	-0.5875
2	RNN	2.9182	7.0631	0.0553
3	Média dos 5 anos	2.9471	7.4624	-0.0545
4	Média do treino	3.1292	7.2674	-0.0001

Melhoria da RNN sobre o baseline '2019 = 2018': -4.42

Fonte: Elaborado pelos autores (2026).

Considerando a concentração de valores próximos de 100%, foi testada uma transformação **logit** para representar os percentuais em outra escala. Para interpretar as previsões, os valores transformados precisaram ser convertidos novamente para a escala percentual por meio da função inversa da transformação utilizada. A **função de ativação linear** foi mantida por ter apresentado melhor resultado nos experimentos anteriores.

Na comparação apresentada a seguir, a transformação logit reduziu o MAE em relação ao experimento inicial e apresentou resultado competitivo frente aos métodos de referência.

Figura 10 – Tabela de comparação dos resultados obtidos com a normalização logit e os métodos de referência

	Modelo	MAE (p.p.)	RMSE (p.p.)	R²
0	RNN	1.9320	7.4496	-0.0509
1	Mediana do treino	1.9429	7.5222	-0.0715
2	Último ano (2018)	2.7946	9.1560	-0.5875
3	Média dos 5 anos	2.9470	7.4624	-0.0545
4	Média do treino	3.1292	7.2674	-0.0001

Melhoria da RNN sobre o baseline '2019 = 2018': 30.87

Fonte: Elaborado pelos autores (2026).

O teste seguinte foi aumentar a **quantidade de épocas** da fase de treino de 04 para 20 para entender o comportamento da rede. Como pode ser visto na imagem abaixo, não se tem ganho na redução do erro com o aumento da quantidade de treinos. A quantidade de épocas 04 foi mantida para os próximos testes.

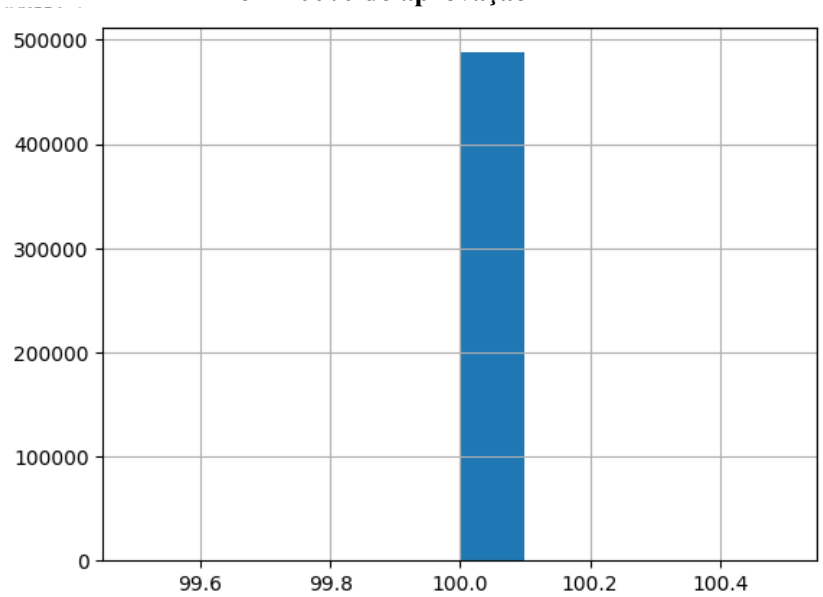
Figura 11 – Utilização de 20 épocas na fase de treinamento

```
Epoch 1/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - loss: 43.1678 - mean_absolute_error: 5.4667 - val_loss: 17.9352 - val_mean_absolute_error: 3.1260
Epoch 2/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 17.1571 - mean_absolute_error: 2.9786 - val_loss: 17.3222 - val_mean_absolute_error: 2.9521
Epoch 3/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.8106 - mean_absolute_error: 2.9033 - val_loss: 17.0555 - val_mean_absolute_error: 2.9627
Epoch 4/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.6859 - mean_absolute_error: 2.8865 - val_loss: 16.9878 - val_mean_absolute_error: 3.0021
Epoch 5/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 16.6385 - mean_absolute_error: 2.8772 - val_loss: 16.9619 - val_mean_absolute_error: 3.0167
Epoch 6/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.6128 - mean_absolute_error: 2.8722 - val_loss: 16.9440 - val_mean_absolute_error: 3.0171
Epoch 7/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 16.5965 - mean_absolute_error: 2.8692 - val_loss: 16.9300 - val_mean_absolute_error: 3.0130
Epoch 8/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step - loss: 16.5853 - mean_absolute_error: 2.8672 - val_loss: 16.9180 - val_mean_absolute_error: 3.0064
Epoch 9/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 16.5747 - mean_absolute_error: 2.8662 - val_loss: 16.9021 - val_mean_absolute_error: 2.9989
Epoch 10/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.5667 - mean_absolute_error: 2.8653 - val_loss: 16.8933 - val_mean_absolute_error: 2.9934
Epoch 11/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 16.5607 - mean_absolute_error: 2.8647 - val_loss: 16.8841 - val_mean_absolute_error: 2.9888
Epoch 12/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.5534 - mean_absolute_error: 2.8643 - val_loss: 16.8774 - val_mean_absolute_error: 2.9833
Epoch 13/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.5498 - mean_absolute_error: 2.8637 - val_loss: 16.8720 - val_mean_absolute_error: 2.9766
Epoch 14/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 16.5471 - mean_absolute_error: 2.8631 - val_loss: 16.8677 - val_mean_absolute_error: 2.9699
Epoch 15/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.5448 - mean_absolute_error: 2.8625 - val_loss: 16.8641 - val_mean_absolute_error: 2.9635
Epoch 16/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.5429 - mean_absolute_error: 2.8619 - val_loss: 16.8610 - val_mean_absolute_error: 2.9575
Epoch 17/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 10s 3ms/step - loss: 16.5411 - mean_absolute_error: 2.8615 - val_loss: 16.8583 - val_mean_absolute_error: 2.9519
Epoch 18/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 16.5394 - mean_absolute_error: 2.8610 - val_loss: 16.8559 - val_mean_absolute_error: 2.9467
Epoch 19/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.5378 - mean_absolute_error: 2.8607 - val_loss: 16.8537 - val_mean_absolute_error: 2.9417
Epoch 20/20
1494/1494 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 16.5363 - mean absolute error: 2.8603 - val loss: 16.8517 - val mean absolute error: 2.9371
```

Fonte: Elaborado pelos autores (2026).

O próximo teste realizado, foi uma nova técnica de limpeza: **Realizar a remoção dos outliers da base**. Porém dada a natureza da distribuição dos dados, a remoção dos outliers gerou uma base representada apenas por sequências de 100% de aprovação. Essa técnica foi descartada para as próximas execuções.

Figura 12 – Histograma após a tentativa de remoção dos valores discrepantes, com concentração em 100% de aprovação



Fonte: Elaborado pelos autores (2026).

O próximo teste foi a **inclusão das features semânticas** dentro do modelo, sendo a primeira feature a informação da rede da escola, que foi normalizada utilizando o método **One-Hot Encoding.**

Com a inclusão da rede, foi obtido uma melhoria próxima de 6%, fator que pode implicar que o feature da rede contém informações úteis para a tarefa.

Figura 13 – Tabela de resultados da SimpleRNN após a inclusão da variável rede de ensino

	Modelo	MAE (p.p.)	RMSE (p.p.)	R^2
0	RNN	1.8109	6.8154	-0.0624
1	Mediana do treino	1.8258	6.8597	-0.0762
2	Último ano (2018)	2.5500	8.0653	-0.4878
3	Média dos 5 anos	2.7215	6.6913	-0.0241
4	Média do treino	2.9687	6.6123	-0.0000

Melhoria da RNN sobre o baseline '2019 = 2018': 28.98%

Fonte: Elaborado pelos autores (2026).

O teste seguinte foi a **substituição da feature de rede pela feature de região,** onde foi obtido um resultado ainda melhor com o MAE de 1.6, em uma melhoria de aproximadamente mais 6% em relação a não inclusão de features semânticas.

Figura 14 – Tabela de resultados da SimpleRNN após a inclusão da variável região

	Modelo	MAE (p.p.)	RMSE (p.p.)	R^2
0	RNN	1.6941	6.7287	-0.0284
1	Mediana do treino	1.7514	6.8625	-0.0697
2	Último ano (2018)	2.5841	8.4748	-0.6313
3	Média dos 5 anos	2.6943	6.7584	-0.0375
4	Média do treino	2.9351	6.6356	-0.0001

Melhoria da RNN sobre o baseline '2019 = 2018': 34.44%

Fonte: Elaborado pelos autores (2026).

O teste seguinte incluiu **simultaneamente as variáveis de região e rede de ensino**. O resultado apresentou pouca variação no MAE e melhora no RMSE e no R^2 em relação a algumas configurações anteriores. Esses resultados sugerem que as variáveis podem contribuir para a previsão.

Figura 15 – Tabela de resultados da SimpleRNN após a inclusão das variáveis região e rede de ensino

	Modelo	MAE (p.p.)	RMSE (p.p.)	R²
0	RNN	1.7078	6.5895	-0.0130
1	Mediana do treino	1.7924	6.7881	-0.0750
2	Último ano (2018)	2.5733	8.2880	-0.6025
3	Média dos 5 anos	2.6807	6.6728	-0.0387
4	Média do treino	2.9440	6.5472	-0.0000

Fonte: Elaborado pelos autores (2026).

Com os melhores parâmetros atingidos sendo: Função de ativação da camada de saída linear, inclusão das features semânticas de rede e região, e normalização logit, avançamos para utilização de uma camada LSTM no lugar da camada simpleRRN e obtivemos o seguinte resultado.

Figura 16 – Tabela de comparação dos resultados da SimpleRNN e dos métodos de referência com as variáveis semânticas

	Modelo	MAE (p.p.)	RMSE (p.p.)	R²
0	RNN	1.7162	6.6113	-0.0197
1	Mediana do treino	1.7924	6.7881	-0.0750
2	Último ano (2018)	2.5733	8.2880	-0.6025
3	Média dos 5 anos	2.6807	6.6728	-0.0387
4	Média do treino	2.9440	6.5472	-0.0000

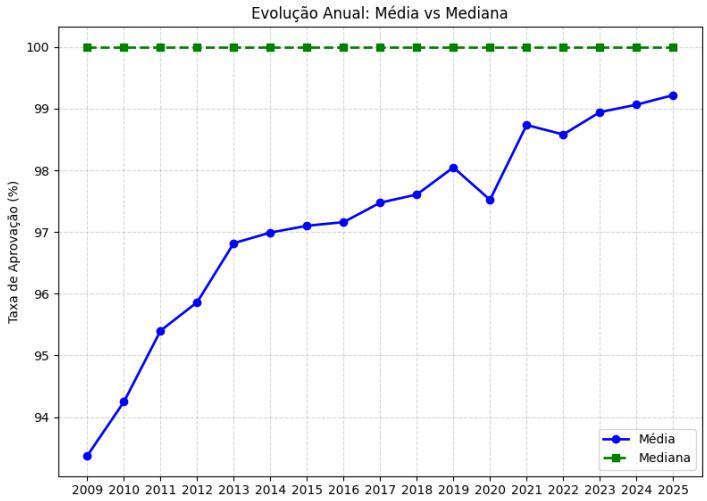
```
Melhoria da RNN sobre o baseline '2019 = 2018': 33.31%
```

Fonte: Elaborado pelos autores (2026).

É percebido que o resultado teve uma pequena piora com a camada **LSTM** para os mesmos parâmetros. O que pode ser explicado pelo tamanho da sequência ser diminuta. A LSTM se destaca por conseguir manter contexto importante em sequências longas, em uma sequência tão curta como a estudada nesse caso, uma simpleRNN já apresenta resultados suficientes utilizando menos recursos computacionais.

Para investigar o efeito do tamanho da sequência histórica, foi realizado um experimento com dados de **2009 a 2025**. A inclusão dos anos da pandemia amplia a variedade de comportamentos presentes na série e pode tornar a previsão mais desafiadora.

Figura 17 – Gráfico da evolução anual da média e da mediana das taxas de aprovação entre 2009 e 2025



Fonte: Elaborado pelos autores (2026).

Figura 18 – Tabela de métricas do modelo SimpleRNN com uma sequência histórica mais longa

	Modelo	MAE (p.p.)	RMSE (p.p.)	R²
0	RNN	0.6200	3.2514	-0.0475
1	Mediana do treino	0.6256	3.2378	-0.0388
2	Último ano (2018)	0.9595	4.0028	-0.5876
3	Média do treino	1.1129	3.1768	-0.0000
4	Média dos 5 anos	2.1967	4.2749	-0.8108

Fonte: Elaborado pelos autores (2026).

Figura 19 – Tabela de métricas do modelo LSTM com uma sequência histórica mais longa

	Modelo	MAE (p.p.)	RMSE (p.p.)	R²
0	RNN	0.6155	3.1863	-0.0060
1	Mediana do treino	0.6256	3.2378	-0.0388
2	Último ano (2018)	0.9595	4.0028	-0.5876
3	Média do treino	1.1129	3.1768	-0.0000
4	Média dos 5 anos	2.1967	4.2749	-0.8108

Fonte: Elaborado pelos autores (2026).

É perceptível que mesmo utilizando sequências maiores e mais complexas, podemos ver que utilizar a rede LSTM teve um ganho marginal pelos indicadores de MAE e RMSE. Porém

o indicador R^2 apresentou uma melhora, implicando que a rede LSTM está desempenhando melhor para outliers neste cenário.

O próximo teste realizado, mantendo a LSTM, foi variar a quantidade de neurônios dentro da rede. Pelo resultado abaixo é visto que essa alteração não implica numa melhor qualidade dos resultados, mostrando que subir a quantidade de neurônios não ajudou a rede de forma considerável a atingir melhores resultados. Dado esse output a quantidade foi mantida em 16, para manter o padrão dos outros testes.

Figura 20 – Tabela comparativa dos resultados ao variar a quantidade de unidades na camada LSTM

	Unidades LSTM	Parâmetros	MAE (p.p.)	RMSE (p.p.)	R^2	Menor val_loss
0	32	5793	0.61307	3.17612	0.00045	8.74669
1	64	19777	0.61313	3.18129	-0.00281	8.78521
2	8	681	0.61484	3.18465	-0.00493	9.07201
3	16	1873	0.61586	3.18707	-0.00646	8.77493

Fonte: Elaborado pelos autores (2026).

O próximo passo foi realizar os testes com a **rede GRU**. Na tabela abaixo podemos ver que não obtivemos alterações significativas no resultado através dessa rede em relação a simpleRNN e a LSTM.

Figura 21 – Tabela de resultados da arquitetura GRU comparada aos métodos de referência

	Modelo	MAE (p.p.)	RMSE (p.p.)	R^2
0	RNN	0.6165	3.1875	-0.0067
1	Mediana do treino	0.6256	3.2378	-0.0388
2	Último ano (2018)	0.9595	4.0028	-0.5876
3	Média do treino	1.1129	3.1768	-0.0000
4	Média dos 5 anos	2.1967	4.2749	-0.8108

Fonte: Elaborado pelos autores (2026).

O teste seguinte foi voltar a base para **apenas 5 anos de sequência** para comparar o comportamento da rede GRU com a LSTM nesse cenário. É perceptível na tabela abaixo que a diferença se fez marginal.

Figura 22 – Tabela de comparação entre GRU e LSTM utilizando uma sequência histórica de cinco anos

	Modelo	MAE (p.p.)	RMSE (p.p.)	R^2
0	RNN	1.7230	6.6228	-0.0232
1	Mediana do treino	1.7924	6.7881	-0.0750
2	Último ano (2018)	2.7644	8.5262	-0.6959
3	Média dos 5 anos	2.8086	7.0041	-0.1444
4	Média do treino	2.9440	6.5472	-0.0000

Fonte: Elaborado pelos autores (2026).

Não foi observada vantagem consistente das arquiteturas LSTM e GRU sobre a SimpleRNN, tornando a SimpleRNN a alternativa com maior custo-benefício entre as arquiteturas avaliadas.

O próximo passo desta atividade foi realizar uma parametrização sistemática de diferentes configurações de rede para averiguar qual apresentaria o melhor custo-benefício. Os resultados podem ser vistos na tabela a seguir.

Tabela 1 – Resultados dos testes com parametrização sistêmica

Rede	Camadas	Neurônios	MAE(p.p.)	Parâmetros	Tempo (s)	Val Loss	Custo Beneficio
SimpleRNN	1	16	0.5887	481	22.9980	8.4230	0.02560
SimpleRNN	1	8	0.5992	177	24.2200	8.5420	0.02474
SimpleRNN	1	32	0.5915	1473	24.6154	8.4084	0.02403
SimpleRNN	1	64	0.5756	4993	28.1069	8.4477	0.02048
LSTM	1	8	0.5927	681	34.4501	8.2579	0.01720
LSTM	1	16	0.5906	1873	35.1947	8.2382	0.01678
LSTM	1	32	0.5894	5793	38.8853	8.2274	0.01516
SimpleRNN	2	16	0.5851	1009	39.1718	8.3313	0.01494
SimpleRNN	2	8	0.5973	313	41.2503	8.4055	0.01448
SimpleRNN	2	32	0.5878	3553	40.8142	8.3761	0.01440
LSTM	1	64	0.5883	19777	43.2728	8.2555	0.01359
SimpleRNN	2	64	0.5913	13249	44.1894	8.4140	0.01338
GRU	1	32	0.5854	4449	45.2661	8.2237	0.01293
GRU	1	16	0.5866	1457	45.8578	8.2363	0.01279
SimpleRNN	3	16	0.5953	1537	52.8433	8.3131	0.01126
GRU	1	64	0.5831	15041	59.8574	8.3213	0.00974
SimpleRNN	3	8	0.5962	449	61.6766	8.3972	0.00967
GRU	1	8	0.5994	537	62.2742	8.2306	0.00962
SimpleRNN	3	32	0.5911	5633	61.7110	8.3418	0.00958
SimpleRNN	3	64	0.5920	21505	62.0766	8.3669	0.00954
LSTM	2	16	0.5927	3985	64.8790	8.2288	0.00914
LSTM	2	8	0.5914	1225	65.7038	8.2643	0.00900
GRU	2	16	0.5895	3089	85.5098	8.2392	0.00689
GRU	2	8	0.5891	969	85.6932	8.3078	0.00687

LSTM	2	32	0.5919	14113	87.1344	8.2634	0.00679
LSTM	3	8	0.5971	1769	88.8708	8.2738	0.00672
GRU	2	32	0.5821	10785	88.4002	8.3344	0.00658
LSTM	3	16	0.5939	6097	92.4579	8.2479	0.00642
LSTM	2	64	0.5918	52801	93.1057	8.2673	0.00636
GRU	2	64	0.5854	40001	93.3432	8.3128	0.00627
LSTM	3	32	0.5936	22433	97.7507	8.2516	0.00607
GRU	3	16	0.5904	4721	112.7363	8.2799	0.00524
GRU	3	32	0.5892	17121	119.9623	8.2962	0.00491
GRU	3	8	0.5991	1401	124.0478	8.6930	0.00483
LSTM	3	64	0.5940	85825	132.7369	8.2849	0.00447
GRU	3	64	0.5903	64961	144.2415	8.3542	0.00409

Para computar qual seria a melhor configuração para o nosso caso, foi gerado o indicador de custo-beneficio dado por MAE/tempo_de_execuação, por esse indicador o melhor resultado foi dado pela rede simpleRNN com uma camada densa e 16 neurônios. Como essa foi o melhor resultado, essa configuração foi escolhida para ser validada contra a base de teste.

Figura 23 – Resultados a rede para a base de testes

```
MAE:  0.6117 p.p.
RMSE: 3.1557 p.p.
R²:   -0.0377

Previsões com erro ≤ 1 p.p.: 90.07%
Previsões com erro ≤ 2 p.p.: 92.41%
Previsões com erro ≤ 5 p.p.: 96.43%
```

Fonte: Elaborado pelos autores (2026).

Como pode ser visto na imagem acima, a rede apresentou um ótimo resultado para a base de teste, demostrando que a assertividade vista nas outras fases se repetiu, logo temos mais uma confirmação que a rede foi capaz de aprender os padrões esperados.

4 CONCLUSÃO

Neste trabalho foi possível experimentar diversas configurações de rede e lidar com um caso real onde a distribuição dos dados se apresentou como desafio. Após vários testes foi percebido a importância de cada parâmetro e como eles podem afetar na precisão da rede. A configuração de rede com melhor custo-benefício foi a simpleRNN com 16 neurônios e função de ativação linear na camada de saída onde aplicamos a normalização logit nos features numéricos e o one-hot-encoding nos features semânticos. Também foi visto que não existe uma correlação linear entre utilizar modelos mais complexos ou aumentar a quantidade de parâmetros, como neurônios e épocas, com a assertividade da rede.